

Cypress Interview Preparation Handbook (2026 Edition)

Volume 1 - Architecture, Custom Commands, Auth, Network Intercept, Retry-ability, Parallelism, Config, API Testing, Cross-Origin, Debugging

Prepared for Srikanth, SDET and Associate Architect QE

September 2026

Contents

How to Use This Handbook	3
SECTION 1 — Architecture & Execution Model (Senior Level)	4
Q1. Explain how Cypress's command queue actually works — why does <code>cy.get(...).click()</code> not execute immediately when the line runs?	4
Q2. Cypress runs tests inside the browser itself, unlike Playwright/Selenium which drive the browser from an external process. What are the real architectural consequences of this?	5
Q3. Explain <code>testIsolation</code> in Cypress 12+ — what changed, and why does it matter for suite design?	5
Q4. How does Cypress's built-in retry-ability differ from Playwright's actionability checks? Where does each one fall short?	6
SECTION 2 — Custom Commands & Reusability (Enterprise Architecture)	7
Q5. How do you design custom Cypress commands so a 500+ test suite doesn't end up with a <code>commands.js</code> file that's 2,000 lines of unrelated logic?	7
Q6. How do you type custom Cypress commands in TypeScript so autocomplete and type-checking work for your whole team?	8
Q7. Design a reusable Page-Object-style pattern in Cypress, given Cypress doesn't ship an official POM abstraction the way some tools do.	9
SECTION 3 — Authentication (<code>cy.session</code> and Beyond)	10
Q8. Explain <code>cy.session()</code> — what problem does it solve, and how is it different from just calling a login command in <code>beforeEach</code> ?	10
Q9. <code>cy.session()</code> is caching a token that expired mid-suite on a long regression run. How do you architect around this?	11
Q10. How do you handle SSO (Okta/Azure AD) login for Cypress tests, given Cypress's historical restrictions on cross-origin navigation?	12
SECTION 4 — Network Interception (<code>cy.intercept</code>)	13
Q11. Explain the difference between using <code>cy.intercept()</code> to stub a response vs. to spy on a real request. When do you use each?	13
Q12. How do you use <code>cy.intercept()</code> to test a feature flag without waiting for the real flag-management system to toggle it?	14
Q13. Simulate a slow network response to test a loading-state UX deterministically in Cypress.	14

SECTION 5 — Retry-ability & Assertions (Real Flakiness Sources)	15
Q14. Explain the difference between implicit and explicit assertions in Cypress, and why mixing them up causes flaky tests.	15
Q15. A test asserts <code>cy.get('.item').should('have.length', 5)</code> and it's flaky — sometimes 4, sometimes 5, sometimes 6. What's actually happening, and how do you fix it properly, not just retry harder?	16
Q16. How do you configure and use custom retry logic (<code>Cypress.config('defaultCommandTimeout')</code> vs. test-level retries) — what's the difference, and when do you touch each?	17
SECTION 6 — Parallel Execution & Cypress Cloud (Top Interview Topic)	18
Q17. How does Cypress achieve parallelization across CI machines, and what does Cypress Cloud actually do that raw CI matrix parallelism doesn't?	18
Q18. Your team can't use Cypress Cloud due to a data-residency/compliance requirement. How do you achieve parallel execution without it?	19
SECTION 7 — Configuration (<code>cypress.config.ts</code>)	19
Q19. How do you structure <code>cypress.config.ts</code> for multiple environments (dev, staging, prod-smoke) cleanly, including environment-specific env variables?	19
Q20. What does <code>setupNodeEvents</code> in <code>cypress.config.ts</code> actually give you access to, and what's a real enterprise use case for it?	20
SECTION 8 — API Testing (<code>cy.request</code>)	21
Q21. How do you architect a reusable API client on top of <code>cy.request()</code> for hybrid API+UI testing, the same way you would with Playwright's <code>APIRequestContext</code> ?	21
Q22. How do you test chained/dependent API calls in Cypress, given <code>cy.request()</code> returns a chainable, not a Promise you can simply <code>await</code> ?	22
SECTION 9 — Cross-Origin & Multi-Tab Limitations (Cypress-Specific Pain Points)	23
Q23. A test needs to verify behavior across two browser tabs opened by the app (e.g., “open in new tab” link). How do you handle this in Cypress, given it can't drive multiple tabs the way Playwright can?	23
Q24. Your app embeds a third-party payment iframe from a different origin. How do you interact with elements inside it in Cypress?	24
SECTION 10 — Debugging & Flaky Test Analysis (Cypress-Specific Tools)	25
Q25. Walk through your systematic process for debugging a Cypress test that's flaky only in CI, never locally.	25
Q26. Two Cypress spec files pass individually but the second one fails when run as part of the full suite, only in <code>cypress run</code> (headless CI mode), not <code>cypress open</code> . What's your hypothesis?	25
RAPID-FIRE REVISION BANK	26
Component Testing	26
CI/CD	27
Cypress-to-Playwright Migration Decisions	27
Coding Challenges (concise prompts)	27
QA Architect Scenarios	28

How to Use This Handbook

This is **Volume 1** of the Cypress companion to your Playwright handbook, built for Senior SDET / QA Architect interviews where Cypress is the primary or secondary automation tool (2025-2026 cycle). Since your Cypress experience is Advanced, this skips fundamentals entirely and goes straight to the architectural and real-incident-level questions interviewers actually ask senior candidates - the command queue model, retry-ability internals, `cy.session`, `cy.intercept`, Cypress Cloud parallelization, and the cross-origin/multi-tab limitations that come up constantly once you've used Cypress at scale.

Every question follows the same format as the Playwright handbook: difficulty, asked-in companies, frequency, interviewer intent, spoken answer, a healthcare-analytics example (matching your Artemis Health background), code, common mistakes, follow-ups, and an architect-level tip.

A **Rapid-Fire Revision Bank** covers Component Testing, CI/CD, Cypress-to-Playwright migration decisions, coding challenges, and architect scenarios for quick recall.

Frequency legend: ★★★★★ Very Frequently Asked · ★★★★ Frequently Asked · ★★★ Occasionally Asked · ★★ Rare but High-Value

SECTION 1 — Architecture & Execution Model (Senior Level)

Q1. Explain how Cypress's command queue actually works — why does `cy.get(...).click()` not execute immediately when the line runs?

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, EPAM, Deloitte Digital **Frequency:** ★★★★★

Why Interviewer Asks This This is the single most fundamental “do you actually understand Cypress, or just use it” question — it explains almost every confusing beginner mistake (chaining, async assumptions, variable assignment) in one concept.

Best Interview Answer Every `cy.*` command doesn't execute synchronously — it enqueues an instruction into Cypress's internal command queue and returns immediately with a chainable object, not a value. Cypress then drains that queue asynchronously, running each command in order, retrying automatically where the command supports retry-ability (like `cy.get()`), and only moving to the next command once the current one resolves. This is why `const el = cy.get('.button')` gives you a chainable wrapper, never the actual element — and why code written as if Cypress were synchronous (`if (cy.get('.button').length > 0)`) simply doesn't work the way people expect coming from Selenium or plain JS.

Real Project Example A cohort-dashboard test that tried `const count = cy.get('[data-testid=cohort-row]').its('length')` and used `count` directly in a JS `if` statement always evaluated against a Cypress chainable object, not a number — the fix was moving the conditional logic inside a `.then()` callback where the resolved value is actually available.

Code Example

```
// WRONG — count is a Cypress chainable, not a number, at this point in the code
const count = cy.get('[data-testid="cohort-row"]').its('length');
if (count > 5) { /* never behaves as expected */ }

// CORRECT — resolved value only exists inside .then()
cy.get('[data-testid="cohort-row"]').its('length').then((count) => {
  if (count > 5) {
    cy.log('Large cohort detected');
  }
});
```

Common Mistakes Trying to use `cy.get()`'s return value directly in synchronous JavaScript logic; assuming `await` works on Cypress commands (Cypress deliberately does not use Promises/async-await for its command chain, by design).

Follow-Up Questions 1. Why did the Cypress team deliberately choose a command queue over native Promises? 2. How does this queue model interact with `cy.then()` vs. `.then()` on a plain Promise elsewhere in the test? 3. What happens if you throw a synchronous error in the middle of building the queue vs. during queue execution?

QA Architect Tip Say explicitly that this queue model is *why* Cypress commands are retryable and why the tool intentionally avoids `async/await` for its own commands — framing it as a deliberate design trade-off, not a limitation, is the senior-level signal here.

Q2. Cypress runs tests inside the browser itself, unlike Playwright/Selenium which drive the browser from an external process. What are the real architectural consequences of this?

Difficulty: Senior / Staff **Asked In:** Microsoft, Adobe, ServiceNow **Frequency:** ★★★★★

Best Interview Answer Because the Cypress test runner executes inside the same run-loop as the application under test, it gets native access to the DOM without serialization overhead — no WebDriver protocol round-trip per command — which is why Cypress commands can feel faster and DOM assertions more reliable for same-origin content. The consequence is Cypress historically couldn't handle true multi-tab or multi-domain scenarios the way Playwright can, because a new tab or a different origin is a genuinely separate browser context that Cypress's in-browser runner isn't naturally embedded into. `cy.origin()` (added later) and Cypress's newer multi-domain support patch specific cross-origin cases, but the fundamental constraint — one test, one browser tab, primarily one origin at a time — still shapes how you architect Cypress test suites differently from Playwright ones.

Real Project Example A cohort-export flow that redirects to a third-party payment/export-service domain to finalize a download required `cy.origin()` to interact with that domain's confirmation page — a scenario that in Playwright would just be a normal `page.goto()` on any origin with zero special handling.

Code Example

```
// Cypress needs explicit cy.origin() to interact with a different domain
cy.visit('/dashboards/cohort/123/export');
cy.origin('https://export-partner.example.com', () => {
  cy.get('[data-testid="confirm-export"]').click();
});
```

Common Mistakes Assuming `cy.origin()` behaves like simply navigating to a URL — it actually creates a sandboxed execution context for that block, and you can't freely pass live DOM references or closures across the boundary, only serializable data.

Follow-Up Questions 1. How would you test a true multi-tab scenario (two users on two tabs) in Cypress vs. how trivially Playwright handles it? 2. What data can and can't cross a `cy.origin()` boundary?

QA Architect Tip This is one of the most common “why did you choose Playwright over Cypress for X” interview threads — be ready to articulate this architectural trade-off precisely, since it's often the actual deciding factor in real migration decisions.

Q3. Explain testIsolation in Cypress 12+ — what changed, and why does it matter for suite design?

Difficulty: Senior **Asked In:** Walmart Global Tech, Razorpay, Flipkart **Frequency:** ★★★★★

Best Interview Answer Before Cypress 12, tests within a spec file shared page state by default unless you explicitly visited a fresh page — a common source of subtle cross-test leakage. From Cypress 12 onward, `testIsolation` defaults to `true`: before every test, Cypress clears cookies, `localStorage`, and `sessionStorage`, and resets to a blank page, giving each test a genuinely clean slate similar in spirit to Playwright's per-test context isolation. This is a deliberate trade-off — safer isolation at a small performance cost (an extra reset per test) — and I keep it enabled except in narrow, deliberate cases where a true multi-step workflow needs to preserve state across chained tests, which I model explicitly rather than disabling isolation suite-wide.

Real Project Example A legacy suite written pre-Cypress-12 had several tests that silently depended on a previous test’s login state persisting — upgrading and enabling `testIsolation: true` immediately surfaced those hidden dependencies as clear, individual test failures instead of a confusing pass/fail pattern that depended on run order.

Code Example

```
// cypress.config.js
module.exports = {
  e2e: {
    testIsolation: true, // default from Cypress 12+
  },
};

// Explicit, intentional state-sharing when truly needed – use cy.session, not disabled is
describe('report builder workflow', () => {
  it('creates a draft report', () => {
    cy.session('report-builder-user', () => {
      cy.loginByApi('analyst');
    });
    cy.visit('/reports/new');
    // ...
  });
});
```

Common Mistakes Disabling `testIsolation` suite-wide to “fix” failures after an upgrade, instead of investigating and fixing the hidden state dependencies that isolation correctly surfaced.

Follow-Up Questions 1. How does `testIsolation` interact with `cy.session()`’s own caching behavior? 2. What’s the performance cost of `testIsolation`, and when would you consciously accept disabling it for a specific spec?

QA Architect Tip Frame `testIsolation` defaulting to `true` as Cypress converging toward the same isolation philosophy Playwright always had — a good answer connects both tools’ evolution toward the same underlying principle.

Q4. How does Cypress’s built-in retry-ability differ from Playwright’s actionability checks? Where does each one fall short?

Difficulty: Senior / Staff **Asked In:** Microsoft, EPAM, Salesforce **Frequency:** ★★★★★

Best Interview Answer Cypress retries the *entire command* (re-querying the DOM, re-evaluating) up to its timeout for commands that support retry-ability — `cy.get()`, `.should()`, and most query commands — but action commands like `.click()` don’t retry the *action itself* the way Playwright’s actionability pipeline does; Cypress mostly relies on the preceding `.should()/assertion` to ensure the element is ready before the action runs once. Playwright’s actionability checks (`attached`, `visible`, `stable`, `enabled`, `receives-events`) are baked into every action command directly, retrying the *action attempt* itself, not just the query before it. In practice this means Cypress tests often need an explicit `.should('be.visible')` before a click where Playwright would handle that transparently inside `.click()` itself.

Real Project Example A “Save” button sliding into place via CSS transition caused occasional missed clicks in Cypress until an explicit `cy.get('[data-testid="save-btn"]').should('be.visible').click()` was added — Playwright’s equivalent `page.getByTestId('save-`

btn').click() needed no such addition, because its actionability check already covers exactly this.

Code Example

```
// Cypress – often needs an explicit readiness assertion before the action
cy.get('[data-testid="save-btn"]')
  .should('be.visible')
  .should('not.be.disabled')
  .click();
```

Common Mistakes Assuming Cypress’s retry-ability means clicks are just as “auto-waited” as Playwright’s — it’s a narrower guarantee, and interviewers specifically probe whether candidates know the difference.

Follow-Up Questions 1. How would you debug a click that “succeeds” in Cypress but doesn’t visibly register in the app? 2. What Cypress config option controls the default retry timeout, and how do you override it per-command?

QA Architect Tip Precisely naming which Cypress commands are retryable (queries, assertions) vs. which aren’t (most actions, by default) is a strong differentiator — vague “Cypress retries everything” answers signal shallow familiarity.

SECTION 2 — Custom Commands & Reusability (Enterprise Architecture)

Q5. How do you design custom Cypress commands so a 500+ test suite doesn’t end up with a commands.js file that’s 2,000 lines of unrelated logic?

Difficulty: Senior / Staff **Asked In:** Walmart Global Tech, Optum, Deloitte Digital **Frequency:** ★★★★★

Best Interview Answer I split custom commands into domain-focused files under cypress/support/commands/ (auth.js, cohorts.js, api.js) imported into one commands.js entry point, rather than one flat file. Each command follows a single-responsibility rule — cy.loginByApi(role) only handles auth, cy.seedCohort(params) only handles data setup — and I avoid commands that do multiple unrelated things internally (login *and* navigate *and* seed data in one call), which becomes untestable and hides what a given test actually depends on.

Real Project Example Splitting a single 2,000-line commands.js into commands/auth.js, commands/cohorts.js, commands/assertions.js, and commands/api.js cut merge conflicts across a 5-engineer team dramatically, mirroring the same Page-Object-splitting problem solved in the Playwright handbook.

Code Example

```
// cypress/support/commands/auth.js
Cypress.Commands.add('loginByApi', (role) => {
  cy.session(role, () => {
    cy.request('POST', '/api/auth/login', {
      username: Cypress.env(`${role}_USER`),
      password: Cypress.env(`${role}_PASS`),
    }).then((response) => {
```

```

        window.localStorage.setItem('authToken', response.body.token);
    });
});
});

// cypress/support/commands/cohorts.js
Cypress.Commands.add('seedCohort', (params) => {
    return cy.request('POST', '/api/cohorts', params).its('body.id');
});

// cypress/support/e2e.js
import './commands/auth';
import './commands/cohorts';

```

Common Mistakes Building “mega commands” that log in, navigate, and seed data all in one call for convenience — this hides what a test actually needs and makes commands impossible to reuse independently.

Follow-Up Questions 1. How would you unit test a custom command in isolation? 2. How do you document custom commands so new team members discover what already exists instead of duplicating logic?

QA Architect Tip Draw the explicit parallel to Playwright fixtures — custom commands are Cypress’s version of the same “reusable, composable, single-responsibility setup” principle, just without built-in dependency injection.

Q6. How do you type custom Cypress commands in TypeScript so autocomplete and type-checking work for your whole team?

Difficulty: Senior **Asked In:** Microsoft, Adobe, EPAM **Frequency:** ★★★★★

Best Interview Answer Custom commands need their type signatures declared by augmenting the global Cypress.Chainable interface in a .d.ts file — without this, TypeScript has no idea `cy.loginByApi()` exists or what arguments it takes, and every custom command call would show a type error despite working at runtime.

Code Example

```

// cypress/support/index.d.ts
declare namespace Cypress {
    interface Chainable {
        loginByApi(role: 'admin' | 'analyst' | 'auditor'): Chainable<void>;
        seedCohort(params: { size: number; name?: string }): Chainable<string>;
    }
}

// cypress/support/commands/auth.ts
Cypress.Commands.add('loginByApi', (role: 'admin' | 'analyst' | 'auditor') => {
    cy.session(role, () => {
        cy.request('POST', '/api/auth/login', {
            username: Cypress.env(`${role.toUpperCase()}_USER`),
            password: Cypress.env(`${role.toUpperCase()}_PASS`),
        });
    });
});

```

```
});  
});
```

Common Mistakes Writing custom commands in TypeScript but forgetting the `.d.ts` augmentation — the command works at runtime but every call site shows a red squiggly, which teams often “fix” by switching to `// @ts-ignore` instead of the actual declaration.

Follow-Up Questions 1. How would you type a command whose return value depends on its input, generically? 2. How does this differ when the project uses `cy-press/support/component.ts` for component testing alongside e2e?

QA Architect Tip Mention keeping the `.d.ts` file colocated and reviewed alongside the command implementation in the same PR — type declarations that drift out of sync with actual command behavior are worse than no types at all.

Q7. Design a reusable Page-Object-style pattern in Cypress, given Cypress doesn't ship an official POM abstraction the way some tools do.

Difficulty: Senior **Asked In:** ServiceNow, Salesforce, Meesho **Frequency:** ★★★★★

Best Interview Answer I use plain JS/TS classes exposing methods that return Cypress chainables — not wrapping every interaction in custom commands, since classes give cleaner composition and typing for this purpose. Each Page Object method should return `cy.get(...)` chains for further chaining, and — same rule as Playwright — assertions belong in the test file, not baked into the Page Object's methods.

Code Example

```
// cypress/pages/CohortExplorerPage.ts  
export class CohortExplorerPage {  
  visit(cohortId: string) {  
    cy.visit(`/dashboards/cohort/${cohortId}`);  
    return this;  
  }  
  
  get memberCount() {  
    return cy.get('[data-testid="cohort-count"]');  
  }  
  
  setAgeRange(range: string) {  
    cy.get('[data-testid="age-range-input"]').clear().type(range);  
    return this;  
  }  
  
  applyFilters() {  
    cy.get('[data-testid="apply-filters-btn"]').click();  
    return this;  
  }  
}
```

```
// cypress/e2e/cohort-explorer.cy.ts  
import { CohortExplorerPage } from '../pages/CohortExplorerPage';
```

```
const cohortExplorer = new CohortExplorerPage();

it('narrows results by age range', () => {
  cohortExplorer.visit('123').setAgeRange('40-60').applyFilters();
  cohortExplorer.memberCount.should('contain.text', 'members');
});
```

Common Mistakes Returning void from Page Object methods instead of this or a chainable, breaking the fluent-chaining pattern that makes this style readable.

Follow-Up Questions 1. How do you handle a Page Object method that needs to wait for an API call before returning? 2. How would you compose smaller “component objects” the same way as the Playwright handbook describes?

QA Architect Tip Note that Cypress’s official guidance has actually shifted away from recommending POM in favor of custom commands + Cypress Testing Library selectors — mentioning this awareness of Cypress’s own documented stance is a strong signal.

SECTION 3 — Authentication (cy.session and Beyond)

Q8. Explain cy.session() — what problem does it solve, and how is it different from just calling a login command in beforeEach?

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, JP Morgan **Frequency:** ★★★★★

Why Interviewer Asks This cy.session() is Cypress’s direct answer to the same auth-performance problem Playwright solves with storageState, and it’s the most commonly asked Cypress-specific auth question in 2025–2026 interviews.

Best Interview Answer cy.session() caches the browser state (cookies, localStorage, sessionStorage) produced by a setup function, keyed by a session ID — the first time a given session ID runs in the process, Cypress executes the setup function fully; every subsequent call with the same ID restores the cached state instantly instead of re-running the setup. This is different from calling a login command in beforeEach, which would re-execute the full login logic (and often a full page load) before every single test, even though the resulting session is identical every time. Combined with testIsolation, cy.session() gives you both a clean per-test slate and near-zero-cost repeated authentication.

Real Project Example Switching 200 dashboard tests from a beforeEach UI login to cy.session('analyst', () => cy.loginByApi('analyst')) cut roughly 3 seconds of login overhead per test down to milliseconds after the first cached run, saving close to 10 minutes across the full suite.

Code Example

```
Cypress.Commands.add('loginByApi', (role) => {
  cy.session(role, () => {
    cy.request('POST', '/api/auth/login', {
      username: Cypress.env(`${role}_USER`),
      password: Cypress.env(`${role}_PASS`),
    }).then((response) => {
      window.localStorage.setItem('authToken', response.body.token);
    });
  });
}, {
```

```

    validate: () => {
      cy.request('/api/session/check').its('status').should('eq', 200);
    },
  });
});

beforeEach(() => {
  cy.loginByApi('analyst'); // cached after the first run in this process
  cy.visit('/dashboards/cohort-explorer');
});

```

Common Mistakes Forgetting the validate option — without it, a cached session that's silently become invalid (expired token) will be reused anyway, causing every subsequent test to fail against a dead session instead of Cypress detecting and refreshing it.

Follow-Up Questions 1. What does the validate callback actually protect against, precisely? 2. How does `cy.session()` scope caching across spec files within the same run vs. across separate cypress run invocations?

QA Architect Tip Explicitly draw the parallel to Playwright's `storageState` + `globalSetup` pattern — interviewers evaluating you against a Playwright-and-Cypress-both background want to hear you recognize it as the same underlying problem solved differently by each tool's architecture.

Q9. `cy.session()` is caching a token that expired mid-suite on a long regression run. How do you architect around this?

Difficulty: Senior / Staff **Asked In:** JP Morgan, Optum, ServiceNow **Frequency:** ★★★★★

Best Interview Answer The validate function is exactly the mechanism for this — it runs every time a cached session is about to be reused, and if it fails, Cypress discards the cache and re-runs the full setup function automatically. I make validate check something that actually proves the session is live against the backend (a lightweight authenticated endpoint), not just check that a token exists in storage syntactically, since a syntactically-present but expired token would pass a naive check and still fail real requests.

Real Project Example A 2-hour regression run against a 45-minute token TTL: without validate, tests failed randomly after the 45-minute mark with confusing, unrelated-looking errors; adding a validate callback hitting `/api/session/check` caused Cypress to transparently re-authenticate exactly once when needed, with zero manual intervention.

Code Example

```

Cypress.Commands.add('loginByApi', (role) => {
  cy.session(role, () => {
    cy.request('POST', '/api/auth/login', { /* ... */ });
  }, {
    validate: () => {
      cy.request({
        url: '/api/session/check',
        failOnStatusCode: false,
      }).then((response) => {
        expect(response.status).to.eq(200);
      });
    },
  });
});

```

```
});  
});
```

Common Mistakes Validating only that a token *string* is present in `localStorage` rather than actually confirming the backend still honors it — this passes even when the token has expired.

Follow-Up Questions 1. How would you handle refresh-token rotation inside the `validate/setup` flow? 2. What's the trade-off of validating on every single test vs. only periodically?

QA Architect Tip Note that this is functionally identical in purpose to the proactive token-refresh worker fixture pattern from the Playwright handbook — same problem, Cypress gives you a built-in hook (`validate`) rather than requiring you to build the check yourself.

Q10. How do you handle SSO (Okta/Azure AD) login for Cypress tests, given Cypress's historical restrictions on cross-origin navigation?

Difficulty: Senior / Staff **Asked In:** Microsoft, Deloitte Digital, Adobe **Frequency:** ★★★★★

Best Interview Answer Same overall strategy as Playwright — avoid driving the full UI redirect for every test — but the mechanics differ because of Cypress's origin restrictions. If the IdP supports a direct token/API-based login (client credentials, or a test-only login endpoint), I authenticate via `cy.request()` directly and inject the resulting session, bypassing the UI redirect entirely, cached with `cy.session()`. If the flow genuinely requires driving the IdP's hosted login page (which lives on a different origin than the app), I use `cy.origin()` to interact with that domain within the same test, since Cypress can't otherwise act on a foreign-origin page in the same command chain.

Code Example

```
// Preferred – API-based, no cross-origin UI interaction needed  
Cypress.Commands.add('ssoLoginByApi', () => {  
  cy.session('sso-user', () => {  
    cy.request('POST', 'https://idp.example.com/oauth/token', {  
      grant_type: 'password',  
      username: Cypress.env('SSO_USER'),  
      password: Cypress.env('SSO_PASS'),  
      client_id: Cypress.env('CLIENT_ID'),  
    }).then((response) => {  
      window.localStorage.setItem('authToken', response.body.access_token);  
    });  
  });  
});  
  
// Fallback – UI-driven, crossing into the IdP's origin  
it('logs in via Azure AD', () => {  
  cy.visit('/');  
  cy.get('[data-testid="sso-login-btn"]').click();  
  cy.origin('https://login.microsoftonline.com', () => {  
    cy.get('#i0116').type(Cypress.env('SSO_USER'));  
    cy.get('#idSIButton9').click();  
    cy.get('#i0118').type(Cypress.env('SSO_PASS'));  
    cy.get('#idSIButton9').click();  
  });  
});
```

```
cy.url().should('include', '/dashboards');
});
```

Common Mistakes Trying to interact with the IdP's hosted login page without `cy.origin()` — Cypress will simply fail to find elements, because the command context never crossed into that origin's execution sandbox.

Follow-Up Questions 1. What data can safely pass into and out of a `cy.origin()` block? 2. How would this whole approach change if the IdP blocks non-interactive password grants entirely?

QA Architect Tip Be ready to explicitly contrast this with Playwright's SSO handling from the other handbook — this cross-origin friction is one of the most concrete, practical reasons some architects choose Playwright over Cypress for SSO-heavy enterprise apps.

SECTION 4 — Network Interception (`cy.intercept`)

Q11. Explain the difference between using `cy.intercept()` to stub a response vs. to spy on a real request. When do you use each?

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, Adobe **Frequency:** ★★★★★

Best Interview Answer `cy.intercept()` with a `body/response` (or `req.reply()`) fully stubs the response, short-circuiting the real backend — used for testing edge cases (empty states, errors) that are hard to reproduce naturally, or for speeding up tests by skipping a slow real call. Used without stubbing, just registering the intercept and later `cy.wait('@alias')`, it acts as a spy — the real network call still happens, and you're just observing/asserting on it (that it fired, its payload, its timing) without changing the outcome. I stub for edge-case and deterministic-timing tests; I spy for “did the app send the right request” contract-style assertions against a real backend.

Real Project Example Testing a dashboard's empty-cohort state: stubbing `cy.intercept('GET', '/api/cohorts', { body: [] })` makes that rare state trivial and instant to test; testing that applying a filter sends the correct query parameters uses a spy-only intercept against the real staging backend, asserting on the captured request URL.

Code Example

```
// Stub — controls the outcome
cy.intercept('GET', '/api/cohorts', { body: [] }).as('getCohorts');
cy.visit('/dashboards/cohort-explorer');
cy.wait('@getCohorts');
cy.get('[data-testid="empty-state"]').should('be.visible');

// Spy — observes the real request
cy.intercept('GET', '/api/cohorts*').as('getCohorts');
cy.get('[data-testid="age-range-input"]').type('40-60');
cy.get('[data-testid="apply-filters-btn"]').click();
cy.wait('@getCohorts').its('request.url').should('include', 'ageRange=40-60');
```

Common Mistakes Stubbing every single network call by default “for speed,” which slowly turns the suite into something that no longer exercises the real backend contract at all — a balance of stubbed edge-case tests and real-integration tests is healthier than either extreme.

Follow-Up Questions 1. How would you assert on a request body for a POST, not just the URL? 2. What's the risk of over-stubbing an entire suite, long-term?

QA Architect Tip Explicitly name the trade-off: stub for determinism and edge-case coverage, spy for real-contract confidence — a suite that's 100% stubbed everywhere has stopped testing integration with the real backend at all.

Q12. How do you use `cy.intercept()` to test a feature flag without waiting for the real flag-management system to toggle it?

Difficulty: Senior **Asked In:** ServiceNow, Atlassian, Meesho **Frequency:** ★★★★★

Best Interview Answer Same pattern as the Playwright handbook's `page.route()` approach — intercept the flags endpoint and return both true and false states across separate tests, validating both UI branches deterministically regardless of the real flag's rollout state in any environment.

Code Example

```
it('shows new comparison UI when flag is enabled', () => {
  cy.intercept('GET', '/api/feature-flags', { cohortComparisonV2: true }).as('flags');
  cy.visit('/dashboards/cohort-explorer');
  cy.wait('@flags');
  cy.get('[data-testid="comparison-v2-panel"]').should('be.visible');
});

it('shows legacy UI when flag is disabled', () => {
  cy.intercept('GET', '/api/feature-flags', { cohortComparisonV2: false }).as('flags');
  cy.visit('/dashboards/cohort-explorer');
  cy.wait('@flags');
  cy.get('[data-testid="comparison-v2-panel"]').should('not.exist');
});
```

Common Mistakes Forgetting `cy.wait('@flags')` after visiting — without it, the assertion can run before the intercepted response has actually been consumed by the app, causing intermittent failures depending on timing.

Follow-Up Questions 1. How would you handle a flag that's read from a bundled config file at build time instead of an API call? 2. How do you clean up flag-mock tests once a flag is fully rolled out?

QA Architect Tip Mention that `cy.wait('@alias')` is your explicit synchronization point here — Cypress doesn't automatically know your assertion depends on that specific network call finishing, so this wait isn't optional, unlike some of Playwright's more implicit waiting.

Q13. Simulate a slow network response to test a loading-state UX deterministically in Cypress.

Difficulty: Senior **Asked In:** Adobe, Microsoft, Walmart Global Tech **Frequency:** ★★★

Best Interview Answer `cy.intercept()` supports a delay option directly on the stubbed response, giving an exact, repeatable delay without relying on real network throttling — the same principle as the Playwright handbook's `delayed.route.fulfill()` pattern.

Code Example

```
it('shows slow-network message after 8 seconds', () => {
  cy.intercept('GET', '/api/cohorts', {
    body: [],
    delay: 9000,
  }).as('getCohorts');

  cy.visit('/dashboards/cohort-explorer');
  cy.get('[data-testid="slow-network-message"]', { timeout: 10000 }).should('be.visible');
});
```

Common Mistakes Using Chrome DevTools network throttling for precise, assertion-driven timing tests instead of `cy.intercept()`'s built-in delay — throttling is better suited to broad exploratory performance checks, not deterministic test assertions.

Follow-Up Questions 1. How would you simulate a request that never resolves at all, to test an infinite-loading fallback? 2. How would you test exponential-backoff retry logic using staged intercepts?

QA Architect Tip Mention tagging deliberately-slow tests separately (a custom Cypress tag or a separate spec folder) so they don't silently inflate the default fast-feedback run — same discipline as the Playwright handbook's equivalent answer.

SECTION 5 — Retry-ability & Assertions (Real Flakiness Sources)

Q14. Explain the difference between implicit and explicit assertions in Cypress, and why mixing them up causes flaky tests.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, EPAM **Frequency:** ★★★★★

Best Interview Answer Implicit assertions are built into a chain via `.should().and()` and are retried automatically as part of Cypress's retry-ability — `cy.get('.el').should('be.visible')` keeps re-querying and re-checking until it passes or times out. Explicit assertions (`expect(value).to.equal(...)` on a value already pulled out via `.then()`) run exactly once, immediately, against whatever value was captured at that instant — no retry. Flakiness happens when someone pulls a value out early with `.then()` and asserts on it explicitly, when the value they actually needed hadn't stabilized yet — the implicit form would have kept retrying and passed once the UI caught up; the explicit form fails immediately on the first, possibly-stale read.

Real Project Example A cohort count check written as `cy.get('[data-testid=cohort-count]').then(($el) => { expect($el.text()).to.equal('250 members') })` failed intermittently because the count element briefly showed a loading placeholder before the real value rendered — rewriting as `cy.get('[data-testid=cohort-count]').should('have.text', '250 members')` let Cypress's retry-ability absorb that natural rendering delay.

Code Example

```
// Explicit assertion — no retry, reads once
cy.get('[data-testid="cohort-count"]').then(($el) => {
  expect($el.text()).to.equal('250 members'); // fails immediately if not yet rendered
});
```

```
// Implicit assertion – retries automatically until it passes or times out
cy.get('[data-testid="cohort-count"]').should('have.text', '250 members');
```

Common Mistakes Defaulting to `.then()` + `expect()` out of familiarity with plain JS testing frameworks (Jest/Mocha style) instead of leaning on Cypress’s retry-ability via `.should()`.

Follow-Up Questions 1. When is an explicit assertion actually the *correct* choice over an implicit one? 2. How would you write a custom `.should()` callback for a complex, multi-field assertion that still benefits from retry?

QA Architect Tip Say plainly: “if I see `.then()` immediately followed by `expect()` in a PR where a `.should()` would work instead, that’s a code-review flag” — this is the Cypress-specific equivalent of the Playwright handbook’s `waitForTimeout` stance.

Q15. A test asserts `cy.get('.item').should('have.length', 5)` and it’s flaky — sometimes 4, sometimes 5, sometimes 6. What’s actually happening, and how do you fix it properly, not just retry harder?

Difficulty: Senior / Staff **Asked In:** JP Morgan, Optum, PhonePe **Frequency:** ★★★★★

Best Interview Answer A fluctuating count almost always means the assertion is racing an in-progress render — items are still being added (or a filter is still being applied) when the check first runs, and depending on exact timing, retry-ability catches it at different intermediate states before eventually timing out or getting lucky. `.should('have.length', 5)` is retried, but if the underlying data genuinely settles at different final counts across runs (e.g., a shared staging environment where other tests are concurrently mutating the same collection), no amount of retrying fixes a fundamentally unstable target. The real fix is either isolating test data (create exactly 5 known items via API, assert against those specific 5, not a global count) or asserting on a stable signal that the render is actually complete before counting.

Real Project Example The “sometimes 4, 5, 6” pattern traced to three other spec files creating and deleting shared staging cohorts concurrently under `fullyParallel`-equivalent execution — the fix was giving each test its own uniquely-named, API-seeded set of items and asserting against those specific IDs, not a raw count of “everything currently in the list.”

Code Example

```
// FRAGILE – depends on total shared state being exactly 5 at assertion time
cy.get('[data-testid="cohort-row"]').should('have.length', 5);
```

```
// ROBUST – asserts against known, test-owned data
const testCohortIds = [];
before(() => {
  for (let i = 0; i < 5; i++) {
    cy.request('POST', '/api/cohorts', { name: `test-${Date.now()}-${i}` })
      .its('body.id').then((id) => testCohortIds.push(id));
  }
});

it('shows exactly the cohorts this test created', () => {
  testCohortIds.forEach((id) => {
    cy.get(`[data-testid="cohort-row-${id}"]`).should('exist');
  });
});
```

Common Mistakes Increasing `.should()`'s timeout or adding `{ retries: 3 }` at the test level as the fix, instead of tracing the count instability back to shared, contended test data.

Follow-Up Questions 1. How would you confirm shared-data contention is the cause vs. a genuine app race condition? 2. How does this connect to the same shared-state debugging approach used in the Playwright handbook?

QA Architect Tip The strongest answer explicitly rejects “just retry more” as a fix and walks straight to data isolation — this is the Cypress-specific instance of the exact same shared-state diagnostic discipline covered at length in the Playwright handbooks.

Q16. How do you configure and use custom retry logic (`Cypress.config('defaultCommandTimeout')` vs. test-level retries) — what's the difference, and when do you touch each?

Difficulty: Senior **Asked In:** Salesforce, Atlassian, Meesho **Frequency:** ★★★

Best Interview Answer `defaultCommandTimeout` controls how long an individual retryable command (like `cy.get()` or `.should()`) keeps retrying before failing — this is about a single command's patience. Test-level retries (configured globally or per-test) controls whether the *entire test* re-runs from scratch if it fails — this is a blunt, whole-test-level safety net, not a substitute for fixing a genuinely flaky assertion. I tune `defaultCommandTimeout` upward only for specific slow operations (rarely globally), and I use retries sparingly in CI as a noise-absorption mechanism, while tracking (not ignoring) which tests actually consume their retry allowance, exactly like the flaky-test-tracking discipline from the Playwright CI handbook.

Code Example

```
// cypress.config.js
module.exports = {
  defaultCommandTimeout: 6000, // default patience for retryable commands
  retries: {
    runMode: 2, // CI: retry a failing test up to 2 times
    openMode: 0, // local interactive debugging: no retries, fail fast
  },
};
```

Common Mistakes Raising `defaultCommandTimeout` globally to “fix” one slow, specific scenario — this makes every genuinely broken assertion elsewhere in the suite take longer to fail, slowing down fast feedback on real bugs.

Follow-Up Questions 1. Why would you set `retries.openMode` to 0 even if `runMode` is 2? 2. How would you track which tests are silently consuming retries in CI, the way you would in a Playwright suite?

QA Architect Tip Draw the explicit distinction between “patience for one command” and “safety net for a whole test” — conflating the two is a common mistake even among experienced Cypress users.

SECTION 6 — Parallel Execution & Cypress Cloud (Top Interview Topic)

Q17. How does Cypress achieve parallelization across CI machines, and what does Cypress Cloud actually do that raw CI matrix parallelism doesn't?

Difficulty: Senior / Staff **Asked In:** Microsoft, Walmart Global Tech, JP Morgan **Frequency:** ★★★★★

Why Interviewer Asks This Cypress's parallelization model is genuinely different from Playwright's shard/worker model, and confusing the two is an immediate signal of surface-level familiarity.

Best Interview Answer Cypress doesn't have Playwright-style in-process workers running multiple tests concurrently on one machine — parallelization happens *across machines*, coordinated by Cypress Cloud (or a compatible open-source alternative). Each CI machine runs `cypress run --record --parallel`, and Cypress Cloud's load-balancer dynamically assigns spec files to whichever available machine finishes its current spec first — not a static, pre-computed split like a naive CI matrix would do. This dynamic balancing matters because spec file runtimes vary a lot; a static split (spec 1-50 to machine A, 51-100 to machine B) can leave one machine finishing in 5 minutes while another takes 20, whereas Cypress Cloud's dynamic queue keeps all machines busy until the whole run finishes.

Real Project Example Splitting a 500-spec-file suite across 8 CI machines with a naive static split (just dividing file count evenly) left total wall-clock time bottlenecked by the slowest machine's uneven share; switching to Cypress Cloud's `--parallel` recording balanced load dynamically and cut total run time by roughly 30% with the same 8 machines.

Code Example

```
# GitHub Actions — each job connects to the same Cypress Cloud run
jobs:
  cypress-run:
    strategy:
      matrix:
        containers: [1, 2, 3, 4, 5, 6, 7, 8]
    steps:
      - run: npx cypress run --record --parallel --ci-build-id ${{ github.run_id }}
```

Common Mistakes Manually pre-splitting spec files by file count or alphabetical order across CI matrix jobs, assuming that's "parallelization" — this is static and load-imbalanced compared to Cypress Cloud's dynamic queue-based balancing.

Follow-Up Questions 1. What happens if one CI machine crashes mid-run under `--parallel`?
2. How would you achieve similar dynamic load-balancing without Cypress Cloud, using an open-source alternative?

QA Architect Tip Precisely explain "dynamic queue-based balancing across machines" vs. Playwright's "in-process worker + shard" model — this contrast is one of the most frequently probed architectural differences between the two tools in senior interviews.

Q18. Your team can't use Cypress Cloud due to a data-residency/compliance requirement. How do you achieve parallel execution without it?

Difficulty: Staff / QA Architect **Asked In:** JP Morgan, Optum, Deloitte Digital **Frequency:** ★★★

Best Interview Answer Without Cypress Cloud's dynamic balancer, I fall back to a static split, but I make it *weighted* rather than naive — profiling historical spec runtimes (from CI logs) and bucketing spec files into roughly equal-duration groups across machines, rather than equal file *counts*, which is what actually causes the imbalance in naive static splits. An open-source alternative like `cypress-split` or a custom script reading a `.json` runtime-history file can approximate Cypress Cloud's balancing without sending data externally.

Real Project Example A compliance-restricted team built a small internal script that read the previous week's CI timing logs, sorted spec files by historical duration descending, and distributed them round-robin across 6 machines (longest-first bin-packing) — this got within about 8% of Cypress Cloud's dynamic balancing efficiency without any external service involved.

Code Example

```
// Simplified weighted static split using historical durations
const specDurations = require('./spec-durations.json'); // { 'cohort.cy.js': 45, ... }
const machineCount = 6;
const buckets = Array.from({ length: machineCount }, () => ({ specs: [], totalTime: 0 }));

Object.entries(specDurations)
  .sort((a, b) => b[1] - a[1]) // longest first
  .forEach(([spec, duration]) => {
    const lightest = buckets.reduce((min, b) => (b.totalTime < min.totalTime ? b : min));
    lightest.specs.push(spec);
    lightest.totalTime += duration;
  });
```

Common Mistakes Splitting purely by file count or alphabetically without any runtime-history weighting — this reproduces exactly the imbalance problem Cypress Cloud's dynamic queue exists to solve.

Follow-Up Questions 1. How would you keep the runtime-history data fresh as specs change over time? 2. What's the failure mode if a new spec file has no historical duration data yet?

QA Architect Tip Show you understand *why* Cypress Cloud's approach works (dynamic > static) even while proposing a compliant workaround — architects are expected to approximate the right behavior under real constraints, not just cite the ideal tool.

SECTION 7 — Configuration (`cypress.config.ts`)

Q19. How do you structure `cypress.config.ts` for multiple environments (dev, staging, prod-smoke) cleanly, including environment-specific env variables?

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, Deloitte Digital **Frequency:** ★★★★★

Best Interview Answer I keep one base config and layer environment-specific `baseUrl` and `env` values via `CYPRESS_ENV`-driven branching or separate `cypress.env.json` files merged

at runtime — never full duplicate config files per environment. Cypress's `env` config option and `Cypress.env()` runtime accessor are the mechanism for injecting environment-specific secrets/URLs without hardcoding them into spec files.

Code Example

```
// cypress.config.ts
import { defineConfig } from 'cypress';

const environment = process.env.CYPRESS_ENV ?? 'staging';

const envConfig = {
  dev: { baseUrl: 'http://localhost:3000', retries: 0 },
  staging: { baseUrl: 'https://staging.example.com', retries: 2 },
  'prod-smoke': { baseUrl: 'https://app.example.com', retries: 2 },
}[environment];

export default defineConfig({
  e2e: {
    baseUrl: envConfig.baseUrl,
    specPattern: environment === 'prod-smoke'
      ? 'cypress/e2e/smoke/**/*.cy.ts'
      : 'cypress/e2e/**/*.cy.ts',
    setupNodeEvents(on, config) {
      return config;
    },
  },
  retries: envConfig.retries,
});
```

Common Mistakes Hardcoding `baseUrl` and secrets directly into spec files, or maintaining three near-duplicate full config files that drift apart over time.

Follow-Up Questions 1. How do you keep secrets out of a committed `cypress.env.json`? 2. How would you override just one config value from the CLI without touching the config file at all?

QA Architect Tip Mention `cypress run --env` and `CYPRESS_`-prefixed environment variables as a CLI-level override mechanism — architects use this for CI-injected secrets without ever committing them to any config file.

Q20. What does `setupNodeEvents` in `cypress.config.ts` actually give you access to, and what's a real enterprise use case for it?

Difficulty: Senior / Staff **Asked In:** ServiceNow, Salesforce, Adobe **Frequency:** ★★★★★

Best Interview Answer `setupNodeEvents` runs in Cypress's Node.js process (not the browser), giving access to the file system, real database connections, and the ability to register custom Node-side tasks callable from spec files via `cy.task()` — this is the escape hatch for anything that genuinely can't happen inside the browser sandbox, like directly seeding a test database, reading/writing files, or generating a signed JWT for auth setup without a real login round-trip.

Real Project Example A `cy.task('seedCohortDirectly', params)` task that writes straight to a test database via a Node database client, bypassing the API entirely for the rare case where

a test needs a cohort in a state the API itself has no endpoint to create (like a specific malformed legacy record used for regression-testing a data-migration bug fix).

Code Example

```
// cypress.config.ts
import { defineConfig } from 'cypress';
import { seedCohortDirectly } from './cypress/tasks/dbTasks';

export default defineConfig({
  e2e: {
    setupNodeEvents(on, config) {
      on('task', {
        seedCohortDirectly(params) {
          return seedCohortDirectly(params); // real DB write, Node-side
        },
      });
      return config;
    },
  },
});

// spec file
cy.task('seedCohortDirectly', { name: 'legacy-format-test', malformed: true }).then((cohort) => {
  cy.visit(`/dashboards/cohort/${cohortId}`);
});
```

Common Mistakes Reaching for `cy.task()` and direct database access as a default data-seeding strategy instead of the API — direct DB seeding bypasses the API's own validation logic, so it should be the exception for scenarios the API genuinely can't produce, not the default.

Follow-Up Questions 1. Why can't you call `cy.task()`'s underlying Node function directly from browser-side test code without this registration? 2. How would you handle a `cy.task()` that needs to be async and takes several seconds?

QA Architect Tip Frame `cy.task()` as a deliberate, narrow escape hatch — overusing it for things the API could do just as well couples your tests to internal database schema instead of the same contract real users go through.

SECTION 8 — API Testing (cy.request)

Q21. How do you architect a reusable API client on top of `cy.request()` for hybrid API+UI testing, the same way you would with Playwright's `APIRequestContext`?

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, Optum **Frequency:** ★★★★★

Best Interview Answer I wrap `cy.request()` calls in domain-specific custom commands (`cy.createTestCohort()`, `cy.deleteTestCohort()`) rather than scattering raw `cy.request('POST', '/api/...')` calls with inline JSON across spec files — same principle as the Playwright handbook's `CohortApiClient` class, just expressed through Cypress's custom-command mechanism instead of a class, since `cy.request()` already returns a chainable that composes naturally with the rest of a Cypress test.

Code Example

```
// cypress/support/commands/cohorts.js
Cypress.Commands.add('createTestCohort', (params) => {
  return cy.request('POST', '/api/cohorts', params).its('body.id');
});

Cypress.Commands.add('deleteTestCohort', (id) => {
  return cy.request('DELETE', `/api/cohorts/${id}`);
});

// spec file – seeding via API, then testing the UI
it('shows correct member count for a seeded cohort', () => {
  cy.createTestCohort({ size: 250, name: `test-${Date.now()}` }).then((cohortId) => {
    cy.visit(`/dashboards/cohort/${cohortId}`);
    cy.get('[data-testid="cohort-count"]').should('have.text', '250 members');
    cy.deleteTestCohort(cohortId);
  });
});
```

Common Mistakes Inlining raw `cy.request()` calls with hardcoded JSON directly in every spec file — any API contract change then requires editing dozens of files instead of one command definition.

Follow-Up Questions 1. How would you handle auth headers being attached automatically to every `cy.request()` call without repeating them everywhere? 2. How do you assert on a `cy.request()` response's JSON schema, similar to the Playwright `ajv` pattern?

QA Architect Tip Point out that `cy.request()` bypasses `cy.intercept()` entirely — it's a real, direct HTTP call from Node/browser context, not something you'd stub — a common point of confusion interviewers probe for.

Q22. How do you test chained/dependent API calls in Cypress, given `cy.request()` returns a chainable, not a Promise you can simply await?

Difficulty: Senior **Asked In:** JP Morgan, ServiceNow, Razorpay **Frequency:** ★★★★★

Best Interview Answer Because Cypress commands aren't Promises, chaining dependent API calls means nesting `.then()` callbacks (or using `cy.request()` calls sequentially, since each waits for the prior command to resolve before running) rather than `async/await` syntax, even though the code can look superficially similar to Promise chains.

Code Example

```
it('report reflects added members', () => {
  cy.request('POST', '/api/cohorts', { name: 'chain-test' }).then((cohortResponse) => {
    const cohortId = cohortResponse.body.id;

    cy.request('POST', `/api/cohorts/${cohortId}/members`, {
      memberIds: ['m1', 'm2', 'm3'],
    }).then(() => {
      cy.request('POST', `/api/cohorts/${cohortId}/report`).then((reportResponse) => {
        expect(reportResponse.body.memberCount).to.eq(3);
      });
    });
  });
});
```

```
});  
});
```

Common Mistakes Trying to use `async/await` directly with `cy.request()` — it silently doesn't work the way it looks like it should, since Cypress commands aren't native Promises, even though they're thenable-adjacent.

Follow-Up Questions 1. Why does Cypress deliberately avoid making its commands real Promises? 2. How would you flatten deeply nested `.then()` chains for readability in a longer sequence?

QA Architect Tip Mention that `cy.wrap()` around a genuinely async external Promise (like a third-party SDK call) is the correct bridge when you truly need Promise-based code inside a Cypress test — not `async/await` on Cypress's own commands.

SECTION 9 — Cross-Origin & Multi-Tab Limitations (Cypress-Specific Pain Points)

Q23. A test needs to verify behavior across two browser tabs opened by the app (e.g., “open in new tab” link). How do you handle this in Cypress, given it can't drive multiple tabs the way Playwright can?

Difficulty: Senior / Staff **Asked In:** Microsoft, Walmart Global Tech, Flipkart **Frequency:** ★★★★★

Why Interviewer Asks This This is the most commonly cited real limitation that pushes teams toward Playwright for specific features — a very realistic, frequently asked scenario question.

Best Interview Answer Cypress fundamentally cannot control more than one browser tab at a time — there's no `context.waitForEvent('page')` equivalent, because the test runner's architecture is built around a single active tab. My workaround is testing the *link/button's attributes* rather than the resulting tab's behavior — asserting the link has `target="_blank"` and the correct `href`, which proves the app *would* open the right thing, without actually needing to control the second tab. If the new-tab content itself genuinely needs functional testing, I test that page's behavior as its own separate, direct `cy.visit()`-based test, decoupled from the “opened via a new tab” mechanism entirely.

Real Project Example A cohort-report “Open in new tab” link: instead of trying to catch and drive the new tab, the test asserts `cy.get('[data-testid="open-report-link"]').should('have.attr', 'target', '_blank').and('have.attr', 'href').then((href) => { cy.visit(href); /* test the report page directly */ })` — proving both the link's correctness and the destination page's correctness, without ever needing two simultaneous tabs.

Code Example

```
it('report link opens correct destination in a new tab', () => {  
  cy.get('[data-testid="open-report-link"]')  
    .should('have.attr', 'target', '_blank')  
    .and('have.attr', 'href')  
    .then((href) => {  
      cy.visit(href as string); // test the destination directly, same tab  
      cy.get('[data-testid="report-title"]').should('be.visible');
```

```
});  
});
```

Common Mistakes Trying to force multi-tab control with browser-launch flags or third-party plugins as a first resort — usually not worth the fragility; the attribute-plus-direct-visit pattern covers the vast majority of real “new tab” test needs.

Follow-Up Questions 1. When would this workaround genuinely not be sufficient, and you’d need to recommend Playwright for that specific test instead? 2. How would you test a scenario where the *original* tab’s state changes as a result of an action taken in the new tab (e.g., a payment confirmation)?

QA Architect Tip Be honest and direct that this is a real Cypress limitation, not something to talk around — architects are expected to know exactly when to recommend Playwright instead of forcing Cypress to do something it fundamentally isn’t built for.

Q24. Your app embeds a third-party payment iframe from a different origin. How do you interact with elements inside it in Cypress?

Difficulty: Senior **Asked In:** Razorpay, PhonePe, Adobe **Frequency:** ★★★★★

Best Interview Answer Same-origin iframes are straightforward — `cy.get('iframe').its('0.contentDoc` Cross-origin iframes (the actual payment-provider case) are where Cypress’s restrictions bite hardest — by default, Cypress cannot reach into a cross-origin iframe’s content at all, because the browser’s own same-origin policy blocks it, and Cypress doesn’t have a native bypass the way Playwright’s `frameLocator` does. The practical answer for cross-origin payment iframes specifically: stub the payment provider’s response via `cy.intercept()` and test your app’s handling of success/failure callbacks, rather than trying to drive the real third-party iframe’s UI — testing a payment provider’s own iframe UI isn’t your application’s responsibility to cover anyway.

Code Example

```
// Testing your app's handling of the payment callback, not the third-party iframe itself  
it('shows success screen after payment provider confirms', () => {  
  cy.intercept('POST', '/api/payments/webhook', { status: 'success' }).as('paymentWebhook')  
  cy.visit('/checkout/123');  
  // Simulate the provider's callback hitting your backend directly,  
  // rather than driving the actual third-party iframe UI  
  cy.request('POST', '/api/payments/webhook', { orderId: '123', status: 'success' });  
  cy.get('[data-testid="payment-success"]').should('be.visible');  
});
```

Common Mistakes Spending significant engineering effort trying to force interaction with a genuinely cross-origin, third-party-controlled iframe instead of testing your own app’s boundary/callback logic, which is the part actually under your control and responsibility.

Follow-Up Questions 1. What Cypress plugin, if any, claims to help with cross-origin iframe access, and what are its real limitations? 2. How would Playwright’s approach to this same scenario differ, concretely?

QA Architect Tip Reframe the question explicitly: “what’s actually my application’s responsibility to test here” — a payment provider’s own iframe UI correctness isn’t your test suite’s job, and recognizing that boundary is the mature answer, not a workaround-hunting one.

SECTION 10 — Debugging & Flaky Test Analysis (Cypress-Specific Tools)

Q25. Walk through your systematic process for debugging a Cypress test that's flaky only in CI, never locally.

Difficulty: Senior / Staff **Asked In:** Every company in this handbook's dataset **Frequency:** ★★★★★

Best Interview Answer First, I ensure CI is capturing video (`video: true`) and screenshots on failure (Cypress's default), and if using Cypress Cloud, Test Replay — which gives a DOM-level, scrubbable replay of the exact failing run, closer to Playwright's trace viewer than a plain video is. Second, the usual CI-specific suspects: headless-vs-headed rendering differences, CI machine resource contention affecting timing-sensitive assertions, and shared-environment data collisions. Third, I check `defaultCommandTimeout` and `pageLoadTimeout` against CI's actual observed latency — CI networks to a staging environment are often slower than a local machine hitting `localhost`.

Real Project Example A CI-only failure on a cohort-filter test traced to Test Replay showing the filter's dropdown menu was still animating open when the click attempted to land — Cypress's retry-ability on `.should('be.visible')` was satisfied by the dropdown being technically visible mid-animation, but the actual click target had shifted; adding `.should('have.css', 'transform', 'none')` as a stability check before clicking fixed it.

Code Example

```
export default defineConfig({
  video: true,
  screenshotOnRunFailure: true,
  defaultCommandTimeout: 8000, // tuned against observed CI latency, not guessed
});
```

Common Mistakes Adding a blanket `cy.wait(1000)` as the first response to a CI-only flaky failure instead of using Test Replay/video to find the actual root cause first.

Follow-Up Questions 1. What does Test Replay give you that a plain video recording doesn't? 2. How would you reproduce a CI-resource-contention issue locally to confirm that hypothesis?

QA Architect Tip Note explicitly that Cypress video is a passive recording (you watch it happen) while Test Replay/trace-style tools let you *inspect DOM state at any point* — knowing this distinction, and which one your team actually has access to, matters for how you'd even approach the debugging.

Q26. Two Cypress spec files pass individually but the second one fails when run as part of the full suite, only in cypress run (headless CI mode), not cypress open. What's your hypothesis?

Difficulty: Senior / Staff **Asked In:** Walmart Global Tech, JP Morgan, PhonePe **Frequency:** ★★★★★

Best Interview Answer Since Cypress runs spec files sequentially within a single browser process by default (not workers-in-parallel the way Playwright can), “passes alone, fails in the full run” usually points to testIsolation not fully protecting against something — a module-level JS variable in a shared support file, a leftover `cy.intercept()` registration that wasn't scoped to just one test, or `localStorage/sessionStorage` keys the app itself sets outside what

testIsolation's reset covers (like indexedDB, which Cypress's isolation historically doesn't clear as thoroughly). The `cypress open` vs. `cypress run` distinction specifically often points to timing — headless mode can run faster or under different resource conditions, exposing a race that interactive mode's slower, watched execution doesn't.

Real Project Example A shared `cy.intercept()` registered in a `before()` hook at the top of one spec file wasn't torn down and, because both spec files ran in the same underlying browser process during a full `cypress run`, silently affected network calls in the second file too — moving the intercept into a `beforeEach` scoped per-test (matching testIsolation's per-test reset philosophy) fixed it.

Code Example

```
// PROBLEM – registered once for the whole file, leaks scope-wise
before(() => {
  cy.intercept('GET', '/api/cohorts', { fixture: 'cohorts.json' });
});

// FIXED – scoped per test, matching testIsolation's per-test reset model
beforeEach(() => {
  cy.intercept('GET', '/api/cohorts', { fixture: 'cohorts.json' });
});
```

Common Mistakes Assuming `testIsolation: true` protects against absolutely everything — it resets cookies/localStorage/sessionStorage and the page, but intercepts, module-level variables in support files, and some browser storage mechanisms need their own explicit per-test scoping.

Follow-Up Questions 1. What exactly does testIsolation reset, and what does it explicitly not touch? 2. How would you confirm whether indexedDB state is the actual culprit?

QA Architect Tip Precisely naming what testIsolation does and doesn't cover — rather than assuming it's a complete reset — is exactly the kind of specific, tool-internals knowledge that separates a Senior from a Staff-level answer here.

RAPID-FIRE REVISION BANK

Quick-recall coverage for topics reserved for a future Volume 2, organized by theme.

Component Testing

Q27. What's the difference between Cypress E2E testing and Cypress Component Testing? ★★★★★ E2E drives the full app through a real browser against a real (or staged) backend; Component Testing mounts an individual React/Vue/Angular component in isolation using Cypress's bundler integration (Webpack/Vite), letting you test a component's props/events/rendering without spinning up the whole app — closer in spirit to Jest + Testing Library, but with a real browser and Cypress's command/retry-ability model.

Q28. When would you choose Component Testing over E2E for a given test? ★★★★★ When the thing under test is a self-contained UI unit (a filter panel, a modal) whose logic doesn't depend on full-app routing/auth/backend — component tests are faster and more isolated; reserve E2E for testing actual user journeys across multiple components and real navigation.

Q29. How do you pass mock props/data into a mounted component test? ★★★★★

```
import { mount } from 'cypress/react';
import { FilterPanel } from '../../src/components/FilterPanel';

it('emits onApply with correct filter values', () => {
  const onApply = cy.stub().as('onApply');
  mount(<FilterPanel onApply={onApply} />);
  cy.get('[data-testid="age-range-input"]').type('40-60');
  cy.get('[data-testid="apply-btn"]').click();
  cy.get('@onApply').should('have.been.calledWith', { ageRange: '40-60' });
});
```

CI/CD

Q30. How do you cache Cypress's binary install across CI runs to speed up pipelines? ★★★★★ Cache ~/.cache/Cypress keyed on the Cypress version in package-lock.json, skipping the binary download step entirely on a cache hit — the single biggest, easiest CI speed win for Cypress pipelines.

Q31. How do you record runs to Cypress Cloud and merge results across parallel CI jobs? ★★★★★ Pass --record --key <project-key> --parallel --ci-build-id <shared-id> from every parallel job; Cypress Cloud automatically aggregates results from all jobs sharing that build ID into one dashboard run — no manual report-merging step needed, unlike Playwright's blob-report merge.

Q32. What's a sensible retries policy for CI vs. local, and why? ★★★★★ runMode: 2 in CI to absorb genuine environment noise; openMode: 0 locally, since retries during interactive debugging just hide the failure you're actively trying to see.

Cypress-to-Playwright Migration Decisions

Q33. What's the single biggest technical driver for migrating a Cypress suite to Playwright? ★★★★★ Multi-tab/multi-origin/cross-browser (especially WebKit/Safari) testing needs that Cypress structurally can't satisfy — if the app genuinely requires those, migration solves a real limitation, not just a preference.

Q34. What's commonly lost or needs re-architecting when migrating cy.session()-based auth to Playwright? ★★★★★ The mental model maps closely (cy.session() → storageState + globalSetup), but Cypress's validate callback pattern needs to become an explicit proactive-refresh fixture in Playwright, since Playwright doesn't have a direct built-in equivalent hook.

Q35. How would you run both suites in parallel during a phased migration, without doubling CI cost indefinitely? ★★★★★ Track a coverage-parity checklist per feature area, decommission the Cypress CI job for a given area only once Playwright coverage is confirmed equivalent — same phased approach as any large test-framework migration, not Cypress/Playwright-specific.

Coding Challenges (concise prompts)

Q36. Write a custom command that seeds a cohort via API and cleans it up after the test, regardless of pass/fail. ★★★★★

```
Cypress.Commands.add('withTestCohort', (params, testFn) => {
  cy.createTestCohort(params).then((id) => {
    testFn(id);
  });
});
```

```
cy.then(() => cy.deleteTestCohort(id)); // runs after testFn's chain completes
});
});
```

Q37. Handle a native browser file download and verify its contents. ★★★★★ Cypress doesn't have a native download API like Playwright's `waitForEvent('download')` — the common pattern is `cy.task()` reading the downloaded file directly from the configured downloads folder on disk, since Cypress can't intercept the OS-level save dialog.

Q38. Test a drag-and-drop reorder within a list. ★★★★★ Use a plugin like `cypress-drag-drop` for real drag simulation, or dispatch native drag events manually (`dragstart/dragover/drop`) via `.trigger()` if avoiding a dependency.

Q39. Assert a table is sorted correctly after clicking a column header, without hard-coding expected values. ★★★★★ Same principle as the Playwright handbook: extract all cell values, compute the expected sorted order programmatically, and assert actual matches computed — never a hardcoded literal array tied to today's data.

QA Architect Scenarios

Q40. Your 400-test Cypress suite takes 2 hours on one CI machine with no budget for Cypress Cloud. How do you cut runtime? ★★★★★ Layered: weighted static parallel split across available CI machines (Q18 pattern), API-driven `cy.session()` auth instead of UI login everywhere, audit and remove blanket `cy.wait(ms)` calls in favor of retryable `.should()` assertions, and tag a fast smoke subset for PR-time feedback with full suite gated to merge/nightly.

Q41. A client requires cross-browser coverage including Safari/WebKit, but your suite is 100% Cypress. How do you scope this without a full rewrite? ★★★★★ Cypress doesn't support WebKit at all — this is a hard limitation, not a config issue. The honest answer: either accept Chromium/Firefox-only coverage and document the gap, or introduce Playwright specifically for the WebKit-required flows as a targeted addition rather than rewriting the whole suite — a hybrid approach is legitimate and common in practice.

Q42. How would you justify to a skeptical manager why Cypress can't easily test a real multi-user real-time collaboration feature, when “the QA team should just figure it out”? ★★★★★ Explain the single-tab-per-test architectural constraint plainly (Section 9, Q23) with a concrete cost estimate for the workaround vs. the cost of introducing Playwright just for that test category — architects present trade-offs with numbers, not just “it's not possible.”

End of Cypress Volume 1. A Volume 2 covering Component Testing in full depth, CI/CD real incidents, and additional broken-code debugging exercises can be generated next in the identical format on request.